

VibeStream

Team Manual: concepts, features and code guide

A music player built to demonstrate Stack, Queue and Linked List

Data Structures mini project - Module 2 (Stack and Queue), Module 3 (Linked List)

C99 engine + Node.js server + HTML / CSS / JavaScript front end

Version 1.2.0

github.com/AlexDsouza107/vibestream | 27 October 2026

Contents

1. The project at a glance **4**

1.1 Two programs, one set of data structures . . . 4

1.2 Syllabus coverage . . . 4

1.3 What the app can do . . . 4

2. Tech stack and how to run it **5**

2.1 Requirements and commands . . . 5

2.2 Folder map . . . 5

3. How the pieces fit together **7**

3.1 Who owns what . . . 7

3.2 Walk-through: what happens when you press Play . . . 8

3.3 The command protocol (one line in, one line out) . . . 8

4. Data-structure concepts used (the heart of the project) **9**

4.0 The building block: the Song record . . . 9

4.1 Singly linked list (Module 3) - Liked Songs . . . 9

4.2 Doubly linked list (Module 3) - playlists . . . 10

4.3 Circular linked list (Module 3) - Repeat All . . . 11

4.4 Stack (Module 2) - listening history . . . 11

4.5 Circular queue (Module 2) - the play queue (array version) . . . 12

4.6 Linked-list queue (Module 2 + 3) - the play queue (dynamic version) . . . 13

4.7 Priority queue (Module 2) - "Play next" . . . 13

4.8 Deque (Module 2) - double-ended queue . . . 14

4.9 Up Next: three structures shown as one list . . . 14

4.10 Shuffle (Module 2 + 3) - Fisher-Yates into a queue . . . 15

4.11 Cost summary . . . 16

5. Feature manual (user guide) **17**

5.1 Playlists and Liked Songs . . . 18

5.2 Up Next (the queue) . . . 20

5.3 Playing your own music . . . 20

5.4 Light mode and Now Playing . . . 21

6. Code guide, file by file **22**

6.1 The data-structure modules . . . 22

6.2 main.c (console program) . . . 22

6.3 engine.c (web program) - sections in order . . . 22

6.4 server.js (64 lines) . . . 23

6.5 web/ (the front end) . . . 23

7. Engine command reference 25

7.1 Try it yourself . . . 25

8. Key flows in more detail 26

8.1 Previous button and the history stack . . . 26

8.2 Uploading a file (browser side) . . . 26

8.3 Reordering the queue . . . 27

8.4 How shuffle and "play from a list" fit together . . . 27

9. Testing, memory safety and design decisions 28

9.1 How the project was checked . . . 28

9.2 Design decisions worth knowing . . . 28

9.3 Limitations and ideas for extension . . . 28

10. Glossary and viva questions 29

10.1 Glossary . . . 29

10.2 Likely viva questions, with short answers . . . 30

1. The project at a glance

VibeStream is a music player. You can browse a playlist, like songs, queue songs, shuffle, make your own playlists and, in the web app, play real audio files from your computer. It was built for a **Data Structures** course, so every important behaviour is deliberately implemented with a classic data structure that you wrote by hand in C (nodes, pointers, `malloc` and `free`) instead of using a library.

1.1 Two programs, one set of data structures

	Console app ("spotify")	Web app ("engine" + browser)
How to start	`make && ./spotify`	`./run.sh`, then open http://localhost:3000
What you see	A numbered text menu	A Spotify / Apple Music style interface
Sound	None: playing prints "Now playing: ..."	Real audio from files you upload (sample songs are silent)
C source	`main.c` + the modules	`engine.c` + the same modules
Best for	Showing the data structures directly in a viva	Demonstrating the finished product

Both programs use exactly the same four modules: `playlist.c`, `liked.c`, `stack.c` and `queue.c`. If you understand those four files, you understand the project.

1.2 Syllabus coverage

Syllabus module	Topics used	Where you will see them
Module 2 Stack and Queue	Stack (array and linked), circular queue, priority queue, double-ended queue (deque), queue using a linked list	Listening history, Up Next queue, "Play next", shuffle order
Module 3 Linked List	Singly linked list, doubly linked list, circular linked list	Playlists (including your own), Liked Songs, Repeat All

> **What we did NOT use**
>
> No trees, graphs, hash tables or sorting algorithms, and no C++ features. The only "extra" algorithm is the Fisher-Yates shuffle (one short loop). The web layer (Node.js, HTML, CSS, JavaScript) is only a front end; none of the data-structure logic lives there.

1.3 What the app can do

- Playlist **VibeStream Mix** with 8 sample songs; add, insert at a position, delete
- **Liked Songs**; create, rename and delete **your own playlists**
- **Shuffle** for any list; **Repeat All** (circular list); Previous / Next
- **Up Next** queue: add, "Play next", remove, clear, drag to reorder
- **Play your own music**: upload MP3 / M4A / WAV / OGG / FLAC (drag and drop works), ID3 tags and cover art are read
- Listening **history** (the Previous button pops it); array or linked-list mode switch for comparing implementations
- Light and dark theme, scroll animations, full-screen Now Playing, volume and seek controls

2. Tech stack and how to run it

Layer	Technology	Why we chose it
Data structures + logic	**C (C99)**, compiled with `gcc` and `make`	The course is about pointers, nodes and manual memory. No external libraries.
Web server	**Node.js 18+** using only built-in modules: `http, fs, path, readline, child_process`	Tiny (64 lines), no `npm install`, easy to read. It only relays messages to the C program.
Messages	**JSON** over HTTP, and plain text lines over a pipe	Simple to debug: you can type commands into the engine yourself.
Interface	**HTML5, CSS3, vanilla JavaScript** (no framework, no build step)	Fewer moving parts; every file is readable. Open it and edit it.
Audio	**HTML5 <audio>**, `Blob` and `URL.createObjectURL`	Plays a local file without uploading it anywhere.
Browser features	CSS custom properties, `backdrop-filter` (glass), `@property`, View Transitions (theme switch), IntersectionObserver (scroll reveal), HTML5 drag and drop, TextDecoder (ID3 tags)	Modern, built in, no packages.
Testing tools	AddressSanitizer / UBSan, macOS `leaks`, a Python script talking to the engine, Chrome DevTools Protocol for browser tests	Memory bugs are the biggest risk in C. Browser tests catch UI regressions.
Version control	Git and GitHub (branches `main`, `ui-redesign`, `v1.2`; tags v1.0.0, v1.1, v1.2.0)	Each UI generation is kept as its own version.

2.1 Requirements and commands

Need	Command	Notes
Build everything	`make`	Builds `spotify` (console) and `engine` (web). Flags: `-std=c99 -Wall -Wextra`; the code compiles with zero warnings.
Run the console app	`./spotify`	Menu-driven. Type the number of an option.
Run the web app	`./run.sh`	Checks gcc and node exist, runs make, starts the server on port 3000. Use `PORT=4000 ./run.sh` to change the port.
Clean	`make clean`	Removes binaries and object files.
Talk to the engine by hand	`printf "play\nstate\n" | ./engine`	Each line in is a command; each answer is one JSON line. Great for understanding the protocol.

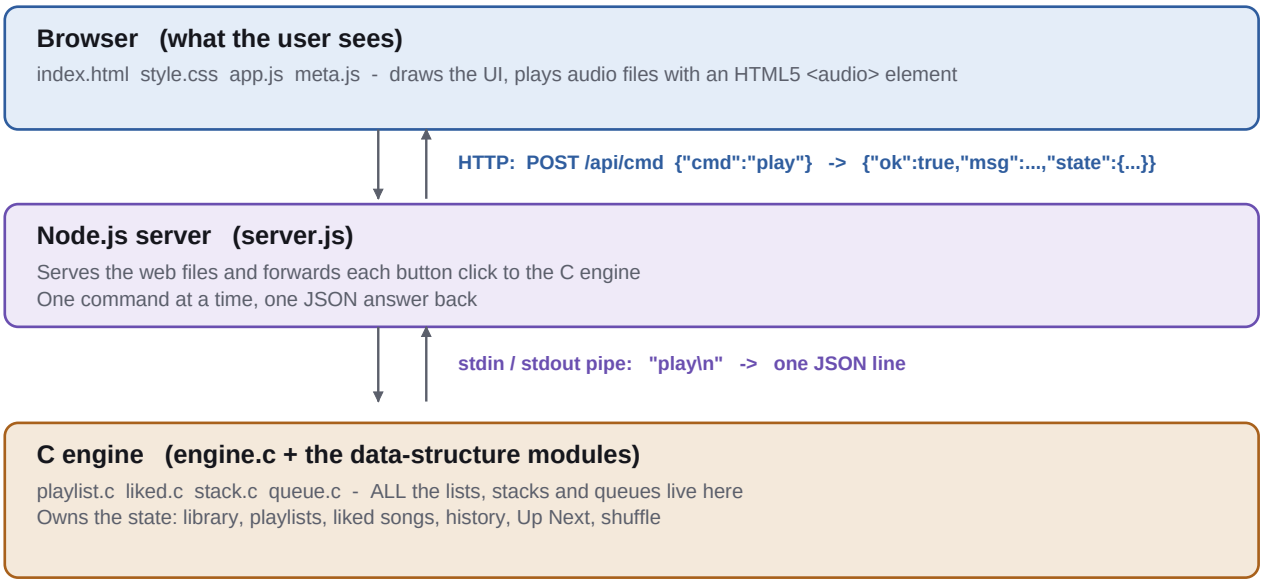
2.2 Folder map

```
spotify/
  song.h              the Song record (id, title, artist, duration)
  playlist.h / .c     DOUBLY linked list + CIRCULAR mode (Repeat All)
  liked.h / .c        SINGLY linked list (Liked Songs) with a play cursor
  stack.h / .c        array stack (max 10) and linked-list stack
  queue.h / .c        circular queue, priority queue, deque, linked-list queue
  main.c              console program: menu, input validation, wiring
  engine.c            the web program: same modules driven by text commands
  server.js           Node web server: serves web/, relays /api/cmd to engine
  web/index.html      page structure          web/style.css   all styling
  web/app.js          UI logic and audio      web/meta.js     file-name / ID3 reader
  web/logo.png ...    logo and icons
  Makefile  run.sh  package.json  README.md
```

File	Lines	Role
engine.c	653	Commands, contexts, shuffle, Up Next editor, JSON output

File	Lines	Role
web/app.js	890	Everything the browser does after the page loads
main.c	395	Console menu program
queue.c / stack.c / playlist.c / liked.c	220 / 102 / 156 / 75	The data structures
web/style.css / index.html / meta.js / server.js	293 / 122 / 135 / 64	Look, layout, tag reader, relay

3. How the pieces fit together



The most important design rule: **the C engine owns the data**. The browser never changes a list itself. It sends a command, the engine changes its linked lists / stacks / queues, and replies with the complete new state. The browser then simply redraws from that state.

3.1 Who owns what

Owned by the C engine	Owned by the browser
The library list, your playlists, Liked Songs	The `<audio>` element and playback position
Listening history (stack)	The link between a song id and its file (`media` map of blob URLs)
Up Next (priority queue, deque, queue)	Cover art blobs from ID3 tags
Shuffle order, Repeat, Shuffle on/off	Volume, theme (light/dark), search text, open view
"Which list am I playing from"	The silent timer used for the sample songs
Array vs linked-list mode	

3.2 Walk-through: what happens when you press Play

1. **You click the Play button**
   app.js calls cmd("play")
2. **Browser sends the command**
   fetch POST /api/cmd with {"cmd":"play"}
3. **server.js forwards it**
   writes "play" + newline to the engine's stdin; remembers who is waiting for the answer
4. **engine.c runs cmd_play()**
   tries Up Next first (pq_dequeue, dq_deleteFront, queue_remove) then the current list (source_next); pushes the song on the history stack
5. **engine prints one JSON line**
   respond(): {"ok":true,"msg":"Now playing: ...","state":{ ...everything... }}
6. **server.js reads that line**
   readline fires, the waiting promise resolves, the JSON goes back to the browser
7. **app.js updates the screen**
   st = d.state; startPlayback() starts the audio file (if any); render() redraws list, player, queue

3.3 The command protocol (one line in, one line out)

```
->  play
<-  {"ok":true,"msg":"Now playing: Blinding Lights - The Weeknd",
     "state":{"dynamic":0,"repeat":0,"shuffle":0,"ctx":0,"capacity":10,
              "playlist":[...], "playlists":[...], "liked":[...],
              "history":[...], "upnext":[...], "upcoming":[...]}}

->  q_move 3|1                      (arguments are separated by |)
<-  {"ok":true,"msg":"ok","state":{...}}

->  list_add 99|2                   (bad id)
<-  {"ok":false,"msg":"That playlist does not exist.","state":{...}}
```

Why return the whole state every time?

It keeps the browser simple and impossible to get out of sync: there is only ever one truth, the engine's. For a playlist of tens of songs the extra JSON is tiny.

4. Data-structure concepts used (the heart of the project)

Each concept below follows the same pattern: **what it is**, **where we use it**, a picture, the real code from the repository, and its cost. The code is copied directly from the source files.

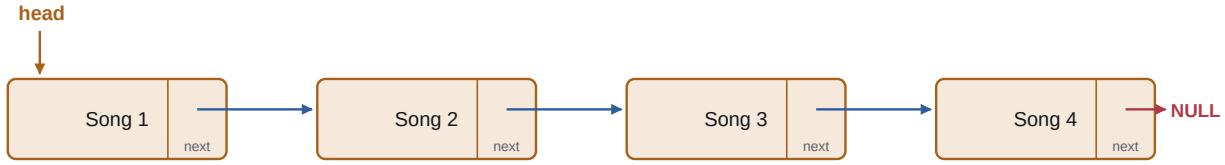
4.0 The building block: the Song record

```
typedef struct {
    int id;
    char title[50];
    char artist[50];
    int duration;   /* length in seconds */
} Song;
```

Every structure stores **copies** of `Song` values (not pointers to one shared object). That is why deleting a song from the library must also remove it from playlists and liked songs by hand (`forget_song()` in engine.c).

4.1 Singly linked list (Module 3) - Liked Songs

A chain of **nodes**. Each node holds a song and one pointer, `next`, to the node after it. You can only walk forward. The last node's `next` is `NULL`. **Used in:** `liked.c`.



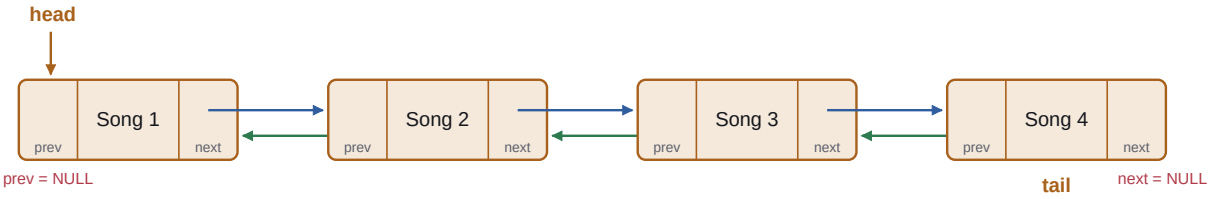

```
int liked_remove(LikedList *l, int id) {
    LikedNode *prev = NULL, *c = l->head;
    while (c && c->song.id != id) {
        prev = c;
        c = c->next;
    }
    if (!c) return 0;
    if (prev) prev->next = c->next; else l->head = c->next;
    if (l->current == c) l->current = prev;       /* keep the cursor valid */
    free(c);
    l->count--;
    return 1;
}
```

The trick when deleting from a singly linked list: keep a `prev` pointer while searching, so you can make `prev->next` skip the removed node. If the removed node is the head, move `head` instead.

Operation	Cost	Why
Add at end (liked_add)	O(n)	Must walk to the last node (there is no tail pointer)
Remove by id	O(n)	Search first, then unlink in O(1)
Next song (liked_next)	O(1)	The play cursor just moves to cursor->next

4.2 Doubly linked list (Module 3) - playlists

Like the singly linked list but every node also has a `prev` pointer, so you can walk **both** directions and unlink a node without searching for its predecessor. The list keeps `head`, `tail` and a `current` (the play cursor). **Used in:** `playlist.c`. The library is one such list and **every playlist you create is its own list**.




```
int pl_insert_at(Playlist *p, int pos, Song s) {
    if (pos < 1 || pos > p->count + 1) return 0;
    PNode *n = new_node(s);
    if (!n) return 0;

    break_circle(p);
    if (!p->head) {                       /* empty list */
        p->head = p->tail = n;
    } else if (pos == 1) {                /* insert at front */
        n->next = p->head;
        p->head->prev = n;
        p->head = n;
    } else if (pos == p->count + 1) {     /* insert at end */
        n->prev = p->tail;
        p->tail->next = n;
        p->tail = n;
    } else {                              /* insert in the middle, before node #pos */
        PNode *c = p->head;
        for (int i = 1; i < pos; i++) c = c->next;
        n->prev = c->prev;
        n->next = c;
        c->prev->next = n;
        c->prev = n;
    }
    p->count++;
    join_circle(p);
    return 1;
}
```

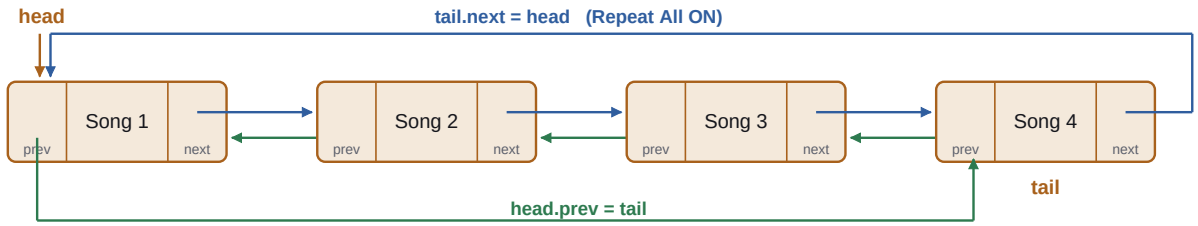
Four cases: empty list, insert at front, insert at end, insert in the middle. In every case **four pointers** must be updated (new node's prev/next and the two neighbours' pointers). Forgetting one is the classic bug.

```
int pl_delete(Playlist *p, int id) {
    break_circle(p);
    PNode *c = p->head;
    while (c && c->song.id != id) c = c->next;
    if (!c) {
        join_circle(p);
        return 0;
    }
    if (c->prev) c->prev->next = c->next; else p->head = c->next;
    if (c->next) c->next->prev = c->prev; else p->tail = c->prev;
    if (p->current == c) p->current = c->prev;   /* keep the cursor valid */
    free(c);
    p->count--;
    join_circle(p);
    return 1;
}
```

Operation	Cost	Why
Insert at end	O(1)	The tail pointer is known
Insert at position p	O(p)	Walk to the position
Delete by id	O(n)	Search; unlinking itself is O(1) thanks to prev
Display backward	O(n)	Start at tail and follow prev

4.3 Circular linked list (Module 3) - Repeat All

A circular list has `tail->next = head` (and, in our doubly version, `head->prev = tail`) so there is **no NULL end**: walking forward never stops. Turning **Repeat All** on joins the ends; turning it off cuts them.




```
void pl_set_repeat(Playlist *p, int on) {
    break_circle(p);
    p->repeat = on;
    join_circle(p);
}

static void break_circle(Playlist *p) {
    if (p->repeat && p->head) {
        p->tail->next = NULL;
        p->head->prev = NULL;
    }
}
static void join_circle(Playlist *p) {
    if (p->repeat && p->head) {
        p->tail->next = p->head;
        p->head->prev = p->tail;
    }
}
```

> **A neat design choice**
>
> Insert and delete code assumes the list ends in NULL. Instead of adding special cases everywhere, `pl_insert_at` and `pl_delete` first call `break_circle()`, do the normal work, then `join_circle()`. Loops that walk the list use `count` (not "until NULL") so they are also safe when circular.

4.4 Stack (Module 2) - listening history

A **LIFO** structure (Last In, First Out): only the top can be touched. We keep every played song on a stack; the song on top is the one playing now. The **Previous** button pops it and plays the song underneath. We wrote it two ways so you can compare them:




```
int astack_push(ArrayStack *s, Song song) {
    if (astack_isFull(s)) return 0;
    s->data[++s->top] = song;
    return 1;
}

int astack_pop(ArrayStack *s, Song *out) {
    if (astack_isEmpty(s)) return 0;
    *out = s->data[s->top--];
    return 1;
}
```

```
int lstack_push(LinkedStack *s, Song song) {
    StackNode *n = malloc(sizeof(StackNode));
    if (!n) return 0;
    n->song = song;
    n->next = s->top;
    s->top = n;
    s->count++;
    return 1;
}

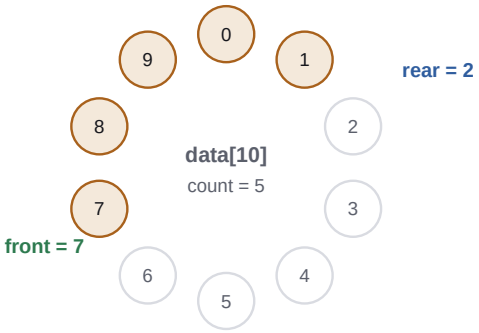
int lstack_pop(LinkedStack *s, Song *out) {
    if (lstack_isEmpty(s)) return 0;
    StackNode *n = s->top;
    *out = n->song;
    s->top = n->next;
    free(n);
    s->count--;
    return 1;
}
```

	Array stack (max 10)	Linked-list stack
Storage	Fixed array `data[10]`, index `top`	Nodes made with malloc, pointer `top`
Full?	Yes, at 10 (`astack_isFull`)	Never (until memory runs out)
push / pop / peek	O(1)	O(1)
Memory	Always reserves 10 songs	Only what is used, plus a pointer per node

When the array stack is full the app drops the **oldest** entry (`astack_drop_oldest`, O(n)) so history keeps working. That is not a pure stack operation, so it is clearly labelled in the code.

4.5 Circular queue (Module 2) - the play queue (array version)

A **FIFO** structure (First In, First Out). A plain array queue wastes space because the front moves right and never returns. A **circular** queue fixes that by wrapping the indexes with the modulo operator.



enqueue(x)

data[rear] = x; rear = (rear + 1) % 10; count++

dequeue()

x = data[front]; front = (front + 1) % 10; count--

The "% 10" wraps the indexes, so space freed at the front is reused. Full: count == 10. Empty: count == 0.

Here the queue holds slots 7, 8, 9, 0, 1: it wrapped past the end of the array.

```
int cq_enqueue(CircularQueue *q, Song s) {
    if (cq_isFull(q)) return 0;
    q->data[q->rear] = s;
    q->rear = (q->rear + 1) % QUEUE_MAX;
    q->count++;
    return 1;
}

int cq_dequeue(CircularQueue *q, Song *out) {
    if (cq_isEmpty(q)) return 0;
    *out = q->data[q->front];
    q->front = (q->front + 1) % QUEUE_MAX;
    q->count--;
    return 1;
}

int cq_removeAt(CircularQueue *q, int i, Song *out) {
    if (i < 0 || i >= q->count) return 0;
    *out = q->data[(q->front + i) % QUEUE_MAX];
    for (int j = i; j < q->count - 1; j++)
        q->data[(q->front + j) % QUEUE_MAX] = q->data[(q->front + j + 1) % QUEUE_MAX];
    q->rear = (q->rear - 1 + QUEUE_MAX) % QUEUE_MAX;
    q->count--;
    return 1;
```

```
}
```

4.6 Linked-list queue (Module 2 + 3) - the play queue (dynamic version)

Same FIFO rule, but built from nodes with `front` and `rear` pointers. Enqueue adds after `rear`; dequeue removes at `front`. No size limit.

```
int lq_enqueue(LinkedQueue *q, Song s) {
    QNode *n = malloc(sizeof(QNode));
    if (!n) return 0;
    n->song = s;
    n->next = NULL;
    if (q->rear) q->rear->next = n; else q->front = n;
    q->rear = n;
    q->count++;
    return 1;
}

int lq_dequeue(LinkedQueue *q, Song *out) {
    if (lq_isEmpty(q)) return 0;
    QNode *n = q->front;
    *out = n->song;
    q->front = n->next;
    if (!q->front) q->rear = NULL;
    free(n);
    q->count--;
    return 1;
}
```

4.7 Priority queue (Module 2) - "Play next"

Elements leave in **priority order**, not arrival order. "Play next" songs are **HIGH**, ordinary queued songs **LOW**. We keep the array **sorted on insertion**, so the next song to leave is always at index 0.



Kept sorted on insert:
HIGH entries first, then LOW.
Equal priority = first come first out.

```
int pq_enqueue(PriorityQueue *q, Song s, int priority) {
    if (pq_isFull(q)) return 0;
    int i = q->count - 1;
    while (i >= 0 && q->data[i].priority < priority) {
        q->data[i + 1] = q->data[i];   /* shift lower-priority entries back */
        i--;
    }
    q->data[i + 1].song = s;
    q->data[i + 1].priority = priority;
    q->count++;
    return 1;
}
int pq_dequeue(PriorityQueue *q, Song *out) {
    if (pq_isEmpty(q)) return 0;
    *out = q->data[0].song;
    for (int i = 1; i < q->count; i++) q->data[i - 1] = q->data[i];
    q->count--;
    return 1;
}
```

4.8 Deque (Module 2) - double-ended queue

A queue you can insert into and delete from at **both** ends. Implemented as a circular array: inserting at the front moves `front` one step *backwards* (with wrap-around).



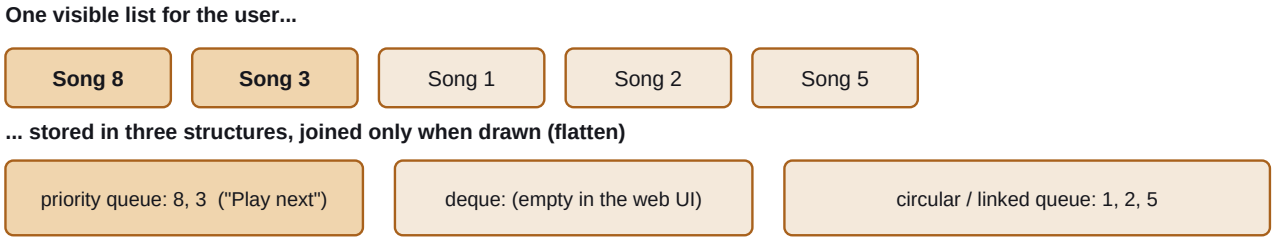

```
int dq_insertFront(Deque *d, Song s) {
    if (dq_isFull(d)) return 0;
    d->front = (d->front - 1 + QUEUE_MAX) % QUEUE_MAX;
    d->data[d->front] = s;
    d->count++;
    return 1;
}

int dq_deleteRear(Deque *d, Song *out) {
    if (dq_isEmpty(d)) return 0;
    *out = d->data[(d->front + d->count - 1) % QUEUE_MAX];
    d->count--;
    return 1;
}
```

In the console app the deque has its own menu entries. In the web app it still sits in the "Up Next" pipeline (it is played after the priority queue) but the web buttons do not expose it directly.

4.9 Up Next: three structures shown as one list

Users see one queue they can reorder. Internally it is the priority queue, the deque and the queue, read in that order (that is also the order they are played). `flatten()` in engine.c copies them into one array for display and reordering.




```
static int upnext_removeAt(int i, Song *out) {
    if (i < 0) return 0;
    if (i < pq.count) return pq_removeAt(&pq, i, out);
    i -= pq.count;
    if (i < deque.count) return dq_removeAt(&deque, i, out);
    return queue_removeAt(i - deque.count, out);
}

static void upnext_move(int from, int to) {
    int n;
    UpItem *a = flatten(&n);
    if (!a || from < 0 || from >= n || to < 0 || to >= n) { free(a); say(0, "Cannot move that song."); return; }
    if (from == to) { free(a); say(1, "ok"); return; }

    int nexts = 0;
    for (int i = 0; i < n; i++) nexts += a[i].next;
    UpItem moved = a[from];
    for (int i = from; i < n - 1; i++) a[i] = a[i + 1];          /* take it out */
    int prefix = nexts - moved.next;                             /* "Play next" block is a prefix */
    /* a "Play next" entry keeps its label while it stays in that block; others join only if dropped strictly
        inside it */
    moved.next = moved.next ? (to <= prefix) : (to < prefix);
    for (int i = n - 1; i > to; i--) a[i] = a[i - 1];             /* make room */
    a[to] = moved;
```

```
    int total_next = 0;
    for (int i = 0; i < n; i++) total_next += a[i].next;
    if (total_next > QUEUE_MAX || (!dynamic && n - total_next > QUEUE_MAX)) {
        free(a);
        say(0, "Array mode holds at most %d songs per queue. Switch to linked-list mode to reorder more.",
            QUEUE_MAX);
        return;
    }
    clear_upnext();
    for (int i = 0; i < n; i++) {
        if (a[i].next) pq_enqueue(&pq, a[i].song, PRIO_HIGH);
        else queue_add(a[i].song);
    }
    free(a);
    say(1, "ok");
}
```

Trade-off to explain in the viva

Removing a song edits just the structure that holds it. **Reordering rebuilds** the structures from the new order: "Play next" entries stay in the priority queue while they remain at the top block, everything else goes into the normal queue. In array mode each queue holds at most 10 songs, so the move is refused with a message if it would not fit.

4.10 Shuffle (Module 2 + 3) - Fisher-Yates into a queue

Shuffle does **not** change your playlist. It makes a random *play order*: copy the songs of the current list into an array, shuffle the array, then **enqueue** them into a linked-list queue. Each Play is simply a dequeue. Every song plays exactly once before the order is rebuilt (or ends).

1. Songs of the list, in order



2. Fisher-Yates: walk from the end, swap each slot with a random earlier one

for i = m-1 down to 1: j = random in [0, i]; swap(arr[i], arr[j])

3. Enqueue the shuffled order into a linked-list queue; Play = dequeue the front




```
static void build_shuffle(int exclude_id) {
    Song *arr;
    int cursor, n = ctx_songs(ctx, &arr, &cursor);
    Song s;
    while (lq_dequeue(&shufq, &s)) { }
    int m = 0;
    for (int i = 0; i < n; i++) if (arr[i].id != exclude_id) arr[m++] = arr[i];
    for (int i = m - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        Song t = arr[i]; arr[i] = arr[j]; arr[j] = t;
    }
    for (int i = 0; i < m; i++) lq_enqueue(&shufq, arr[i]);
    free(arr);
    shuf_dirty = 0;
}

static int shuffle_next(Song *out, int exclude_id) {
    if (shuf_dirty) build_shuffle(exclude_id);
    if (lq_dequeue(&shufq, out)) return 1;
    if (repeat_on && ctx_count(ctx) > 0) {                    /* Repeat All: reshuffle and go again */
        build_shuffle(exclude_id);
        return lq_dequeue(&shufq, out);
    }
    return 0;
}
```

Fisher-Yates in one sentence

Go from the last position down to position 1; swap each element with a randomly chosen element at or before it. It produces every ordering with equal probability in O(n), unlike "swap two random elements many times".

4.11 Cost summary

Structure	Insert	Remove	Access / next	Memory style
Singly linked (liked)	O(n) end	O(n)	O(1) with cursor	malloc per node
Doubly linked (playlist)	O(1) end, O(p) at p	O(n) search + O(1) unlink	O(1) next / prev	malloc per node
Circular linked	same as doubly	same	O(1) wrap-around	no extra memory
Array stack	O(1)	O(1)	O(1) peek	fixed 10
Linked stack	O(1)	O(1)	O(1) peek	malloc per node
Circular queue	O(1)	O(1); removeAt O(n)	O(1) front	fixed 10
Linked queue	O(1)	O(1); removeAt O(n)	O(1) front	malloc per node
Priority queue (sorted array)	O(n) (shift)	O(n) (shift)	O(1) peek	fixed 10
Deque (circular array)	O(1) both ends	O(1) both ends	O(1)	fixed 10
Shuffle (Fisher-Yates + queue)	O(n) build	O(1) per song	-	one copy of the list

5. Feature manual (user guide)

Every feature below lists **how to use it**, **which data structure drives it** and **where the code is**, so a demo and a code question can be answered together.

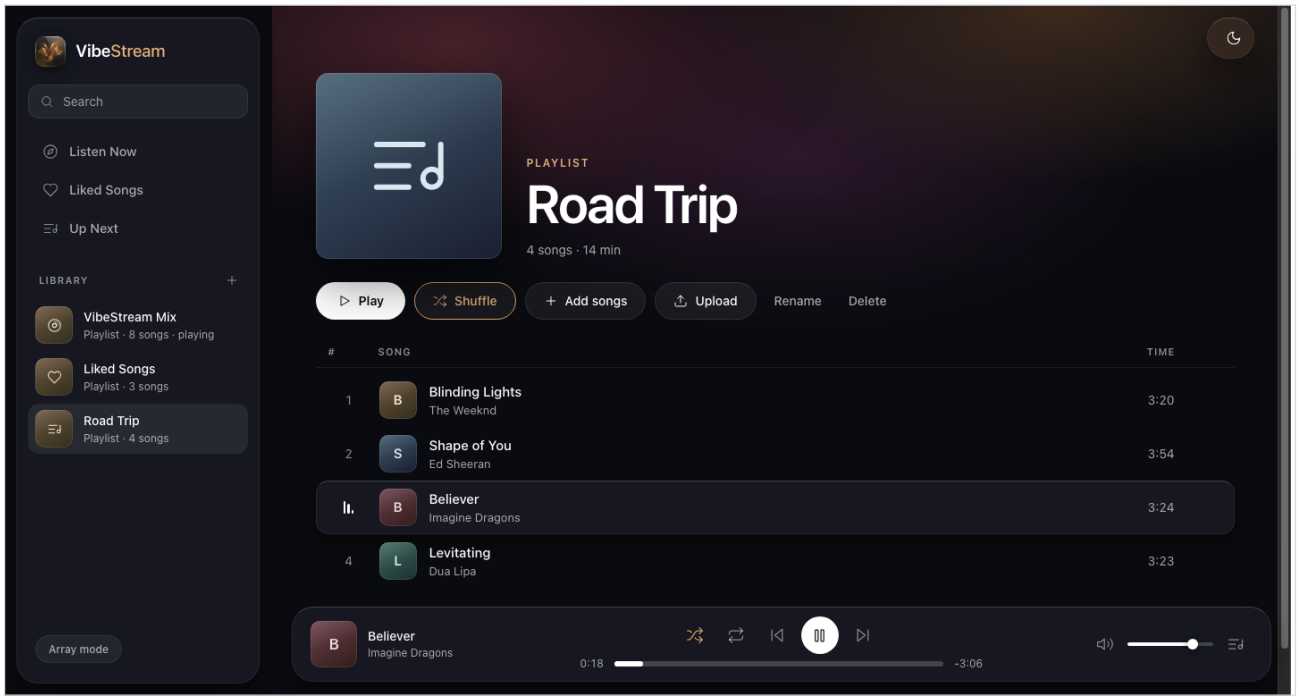


The home screen: sidebar with your library, song list, floating player. The orange pip marks the open page.

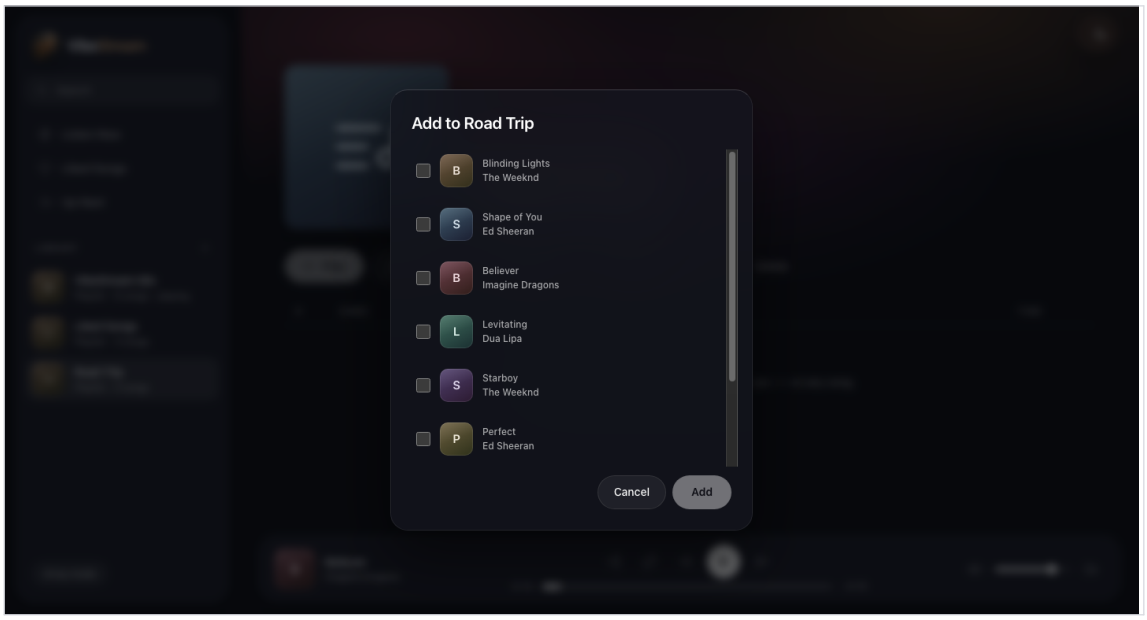
Feature	How to use it	Data structure behind it	Code
Play a song	Double-click a row, or hover and press its play icon	Pushes onto the history stack; moves the list cursor	engine.c play_id, play_song
Play / Pause	Big Play button, the player button, or the Space bar	Pause is browser-only (audio element)	app.js togglePlay
Next	Next button	Priority queue, then deque, queue, then the list (shuffled or in order)	engine.c cmd_play
Previous	Previous button (restarts the song first if more than 3 s in)	Stack: pop the current song, peek the one below	engine.c cmd_prev
Repeat All	Repeat button	Circular linked list: ends joined	playlist.c pl_set_repeat
Shuffle	Shuffle button beside Play, or in the player bar	Fisher-Yates into a linked-list queue	engine.c build_shuffle
Like a song	Heart in Now Playing, or ... menu > Save to Liked Songs	Singly linked list	liked.c liked_add
Create a playlist	+ next to Library, type a name	A new doubly linked list per playlist	engine.c list_new
Add to a playlist	Add songs button, or ... menu > Add to playlist	Insert at the end of that list	engine.c list_add
Up Next	Queue button in the player	Priority queue + deque + circular/linked queue	engine.c flatten
Reorder queue	Drag a song, or focus its grip and use Arrow Up / Down	Rebuild from the flattened array	engine.c upnext_move
Add your own audio	Add song button, or drag files onto the list	Becomes a normal library node; file stays in the browser	app.js addFiles, meta.js
Array / Linked mode	Button at the bottom of the sidebar	Swaps history + queue between array and linked versions	engine.c toggle_mode
Light / dark theme	Sun / moon, top right	(browser only)	app.js paintTheme

5.1 Playlists and Liked Songs

The sidebar **Library** lists three kinds of list: the main **VibeStream Mix** (every song), **Liked Songs**, and the playlists you create. Click one to open it. Each has the same Play / Shuffle buttons; your own playlists also have Add songs, Upload, Rename and Delete.

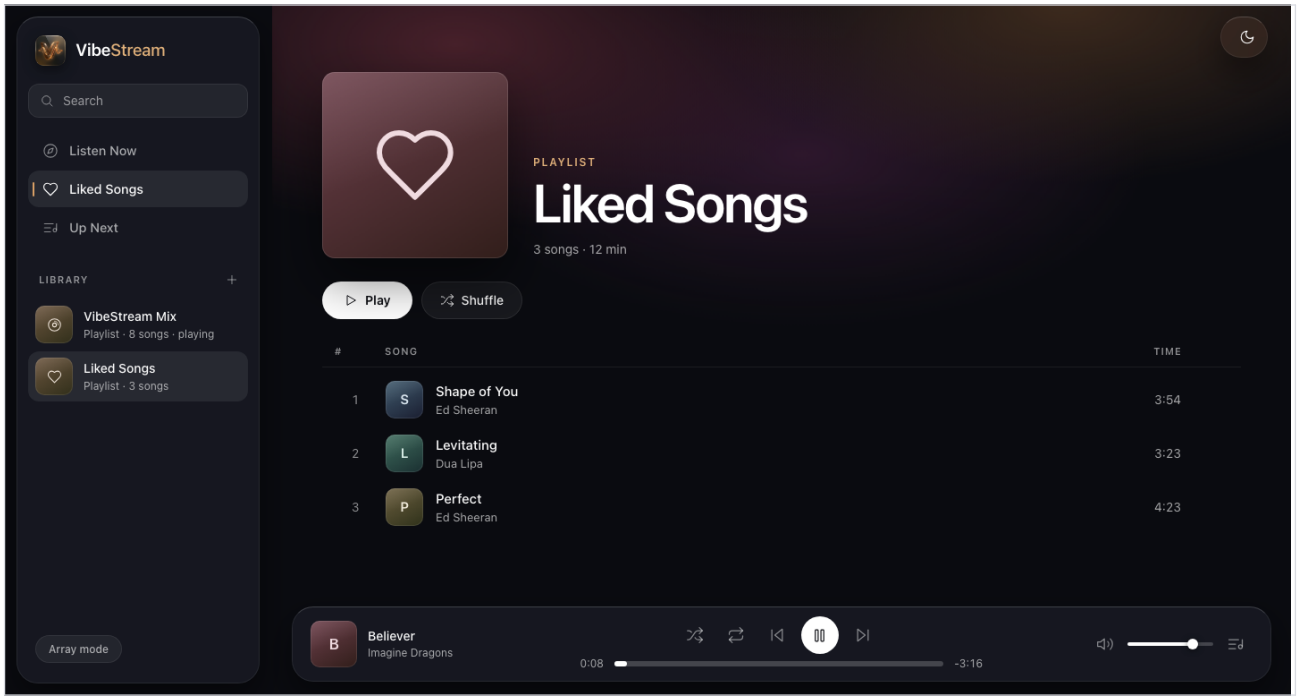


A user-made playlist ("Road Trip") with Shuffle switched on (copper outline). Highlighted row = now playing.



Add songs opens a checklist of library songs not yet in the playlist.

- **Deleting a song from the library** removes it from every playlist and from Liked Songs. **Deleting a playlist** never deletes songs.
- Duplicate songs inside one playlist are rejected (`pl_find` check in `list_add`).
- Playback always continues **inside the list you started from**: the engine remembers it as `ctx` (0 = library, 1 = liked, 2 and up = your playlists).



Liked Songs is a singly linked list; it plays in the order the songs were liked.

5.2 Up Next (the queue)



The Up Next drawer: Now playing, the editable queue ("Play next" badge on top), Add songs, and a preview of what plays after the queue.

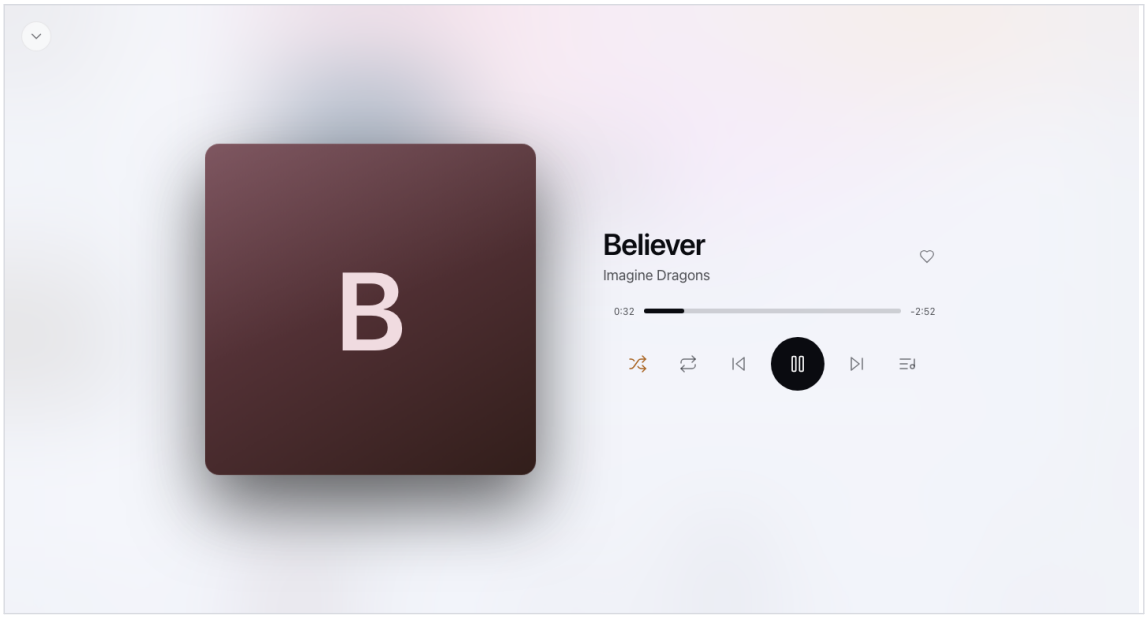
- **Drag** a song up or down; a white line shows where it will land. **Keyboard:** Tab to the grip handle, then Arrow Up / Down.
- **x** removes one song; **Clear** empties the queue; **Add songs** opens the picker.
- Array mode allows at most 10 songs per queue (the drawer tells you when full). Switch to Linked list mode (sidebar) to queue more.
- The order is real: pressing Next plays the top item (we tested this automatically).

5.3 Playing your own music

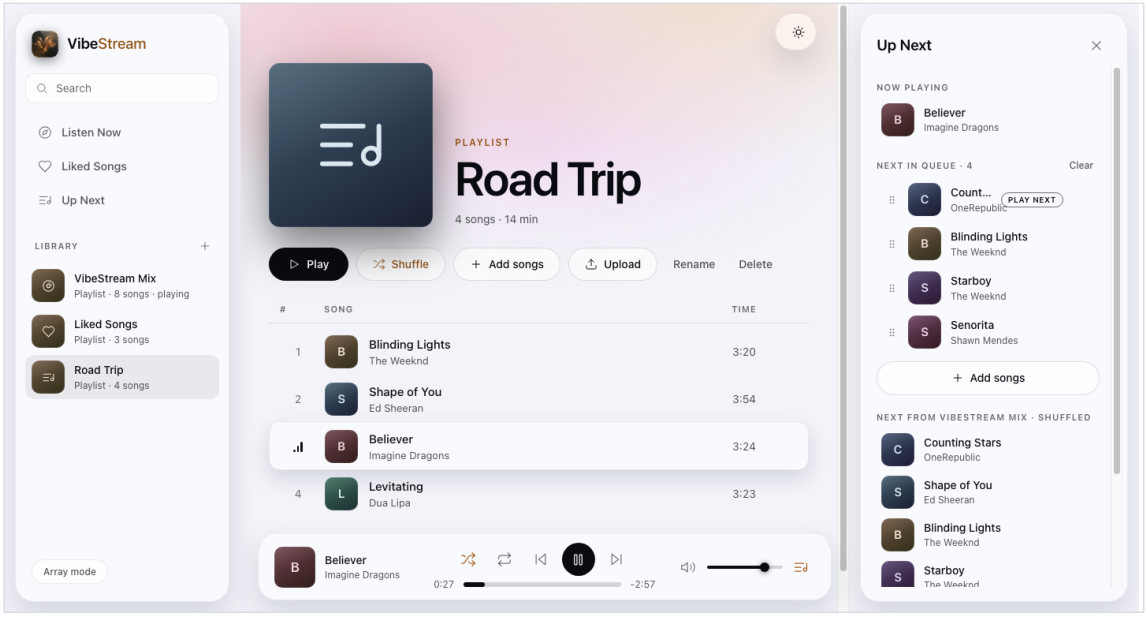
- Press **Add song** (library) or **Upload** (playlist), or drag audio files onto the song area. A dashed overlay confirms the drop zone.
- Supported: MP3, M4A, AAC, WAV, OGG, Opus, FLAC (what your browser can decode). Other files are ignored.
- Title and artist come from the MP3's ID3 tags; otherwise from the file name: `Artist - Title.mp3`, else artist "Local Track". Embedded cover art is shown; files without art get a vinyl-disc icon.
- Duration is read from the file and shown as m:ss. The scrubber supports click, drag and arrow keys; volume and mute are in the player bar.
- Files never leave your computer. They are held as temporary `blob:` addresses and freed when you remove the track.

> **Known limitation**
>
> Uploaded files are **not saved**. If you reload the page the uploaded tracks are removed from the library (the app tells you). The sample songs and the manual "Insert manually" songs have no audio file, so they play on a silent timer.

5.4 Light mode and Now Playing



Light theme in the full-screen Now Playing view. The switch uses a circular reveal animation from the sun / moon button.



Up Next in light mode.

6. Code guide, file by file

Read the files in this order: `song.h` -> `playlist.c` -> `stack.c` -> `queue.c` -> `liked.c` -> `main.c` -> `engine.c` -> `server.js` -> `web/app.js`.

6.1 The data-structure modules

File	Public functions	What to remember
playlist.c	pl_init, pl_insert_end, pl_insert_at, pl_delete, pl_find, pl_display_forward / backward, pl_next, pl_previous, pl_reset_cursor, pl_set_repeat, pl_free	Doubly linked. `current` is the play cursor. Repeat mode is a circle made by break_circle / join_circle. Loops use `count`.
liked.c	liked_init, liked_contains, liked_add, liked_remove, liked_next, liked_reset_cursor, liked_display, liked_free	Singly linked. Duplicates are rejected. Removing the cursor node moves the cursor back so it never dangles.
stack.c	astack_* (init, push, pop, peek, isEmpty, isFull, size, drop_oldest, display) and lstack_* (init, push, pop, peek, isEmpty, size, display, free)	Two parallel implementations with the same names. Pop/peek return 0 when empty instead of crashing.
queue.c	cq_* circular queue, pq_* priority queue, dq_* deque, lq_* linked queue; each has removeAt (added for the Up Next editor)	Sizes capped by `QUEUE_MAX = 10` for the array versions. All functions return 1 on success, 0 on failure.
song.h	struct Song, song_print	The only type shared by everything.

6.2 main.c (console program)

- App struct holds every structure. `read_line` and `read_int` validate all input (clear the buffer, reject bad numbers, handle Ctrl-D).
- `do_play`: priority queue -> deque -> queue -> playlist. `do_previous`: pop then peek on the history stack.
- `toggle_mode` moves data between the array and linked versions. `cleanup` frees every node on exit.

6.3 engine.c (web program) - sections in order

Section	Key functions	Purpose
Contexts	ctx_list, ctx_valid, ctx_count, ctx_songs, reset_cursors	Which list we are playing from (library, liked, user playlist); copy a list to an array
History and queue helpers	hist_record, hist_pop, queue_add, toggle_mode	Pick the array or linked version depending on `dynamic`
Up Next	flatten, clear_upnext, upnext_removeAt, upnext_move	One visible list over three structures
Shuffle	build_shuffle, shuffle_next	Fisher-Yates into a linked queue
JSON output	json_str, json_song, json_list, json_upcoming, print_state, respond	Builds the answer line (escapes quotes and backslashes)
Commands	handle, cmd_play, cmd_prev, cmd_list, source_next, play_song	Parse "command arg|arg" and run it
main	sample songs, command loop	Preloads 8 songs; one input line -> one JSON line; frees everything at end

Play order, as code

```
static void cmd_play(void) {
    Song s, buf;
    if (pq_dequeue(&pq, &s)) play_song(&s, " (from Up Next)");
    else if (dq_deleteFront(&deque, &s)) play_song(&s, " (from Up Next)");
    else if (queue_remove(&s)) play_song(&s, " (from Up Next)");
    else {
        const Song *n = source_next(&buf);
        if (n) play_song(n, "");
        else if (ctx_count(ctx) == 0) say(0, "Nothing to play: this list is empty.");
        else {
            reset_cursors();
            shuf_dirty = 1;
            say(0, "End of list. Press Play to start again, or turn on Repeat All.");
        }
    }
}

static void cmd_prev(void) {
    if (hist_size() < 2) { say(0, "No previous song in history."); return; }
    Song cur, prev;
    hist_pop(&cur);
    hist_peek(&prev);
    say(1, "Now playing: %s - %s (previous, popped from history stack)", prev.title, prev.artist);
}
```

Choosing the next song from the current list

```
static const Song *source_next(Song *buf) {
    if (shuffle_on) return shuffle_next(buf, current_id()) ? buf : NULL;
    if (ctx == 1) {
        const Song *s = liked_next(&liked);
        if (!s && repeat_on) { liked_reset_cursor(&liked); s = liked_next(&liked); }
        return s;
    }
    Playlist *p = ctx_list(ctx);
    return p ? pl_next(p) : NULL;              /* circular lists wrap by themselves in Repeat All */
}
```

6.4 server.js (64 lines)

- Starts `engine` once with `child_process.spawn`.
- Because the engine answers **exactly one line per command**, the server keeps a first-in-first-out list of waiting requests (`waiting[]`) and resolves them in order. That keeps simultaneous clicks safe.
- Serves files from `web/` with a path-traversal guard (a request for `../server.js` returns 404).

6.5 web/ (the front end)

File	What is inside
index.html	Static page skeleton: sidebar, main area, queue drawer, floating player, Now Playing overlay, dialogs (add manually, name, song picker), hidden file input. Every interactive element has an `id` used by app.js.
style.css	Design tokens at the top (`:root[data-theme=dark|light]`: colours, glass fill, borders). Then glass material, layout, rows, player, queue, dialogs, responsive rules, reduced-motion rules.
meta.js	`parseFileName`, `readDuration` (detached Audio element), `readID3` (ID3v2.2 / 2.3 / 2.4: title, artist, cover picture). No dependencies; also loads in Node for testing.
app.js	Numbered sections: 1 State, 2 Icons, 3 Helpers, 4 Media library (blob URLs), 5 Engine calls, 6 Audio engine (play, pause, seek, volume, scrubbers), 7 Rendering (library, view, player, queue, menu), 7b Playlists and picker, 8 Uploading, 9 Scroll animation, theme, navigation.

The one function every action goes through

```
async function cmd(text, opts) {
  try {
    const r = await fetch('/api/cmd', { method: 'POST', body: JSON.stringify({ cmd: text }) });
    const d = await r.json();
    const started = d.ok && d.msg.startsWith('Now playing');
    st = d.state;
    if (started) startPlayback();
    else if (!d.ok && /End of playlist|Nothing to play/.test(d.msg)) setPlaying(false);
    if (!started && !(opts && opts.quiet && d.ok)) toast(d.msg, d.ok);
    render();
    return d;
  } catch (e) {
    toast('Cannot reach the server. Is it running?', false);
  }
}
```

`cmd` sends the command, stores the new state in `st`, starts audio if a new song began, shows a message, then calls `render()`. Almost every button in the UI is just `cmd("...")`.

Starting a real audio file

```
function startPlayback() {
  const cur = current();
  elapsed = 0;
  playing = true;
  if (cur && media[cur.id]) {
    switching = true;
    audio.src = media[cur.id].url;
    audio.currentTime = 0;
    audio.play().catch(onPlayError).finally(() => { switching = false; });
  } else {
    audio.pause();            // simulated song: no sound, the clock below drives the bar
  }
}
```

> **Security habit used throughout app.js**
>
> Song titles and file names are shown with `textContent`, never `innerHTML`, so a song called `<img onerror=...>` cannot run code. Icons (our own static strings) are the only things set as HTML.

7. Engine command reference

Type these into `./engine` (one per line) to experiment. **ID** = song id, **L** = list id (0 library, 1 liked, 2+ playlists).

Command	Arguments	What it does	Structure
state	-	Return the current state, change nothing	-
play	-	Play the next song (Up Next first, then the current list)	pq / deque / queue, list cursor, history stack
prev	-	Go back one song	history stack pop + peek
play_id	ID|L	Play that song and make L the current list	cursor move, history push
play_ctx	L	Play list L from its start (random start if shuffle is on)	cursor reset / shuffle queue
shuffle	-	Toggle shuffle	linked queue rebuilt lazily
repeat	-	Toggle Repeat All on every list	circular linking
mode	-	Switch array / linked-list versions	stack + queue migrate
pl_add	title|artist|secs	Add a song to the end of the library	doubly linked insert at tail
pl_insert	pos|title|artist|secs	Insert at a position	doubly linked insert
pl_delete	ID	Delete from the library and every other list	unlink + free
like / unlike	ID	Add / remove from Liked Songs	singly linked list
list_new	name	Create a playlist (reply contains "(id N)")	new doubly linked list
list_rename / list_delete	L|name / L	Rename / delete a playlist	free the list
list_add / list_remove	L|ID	Add / remove a song in a playlist	insert at end / unlink
q_add	ID	Add to the end of Up Next	circular or linked queue enqueue
playnext	ID	Add to the top of Up Next (HIGH)	priority queue enqueue
q_remove_at	index	Remove the item at that position of Up Next	removeAt on the right structure
q_move	from|to	Move an Up Next item	flatten, move, rebuild
q_clear	-	Empty Up Next	dequeue until empty
q_remove, pq_remove, dq_delfront, dq_delrear, pq_low, dq_front, dq_rear	ID or -	Direct access to each structure (used by the console app and for demos)	queue family

7.1 Try it yourself

```
$ printf 'shuffle\nplay_ctx 0\nplay\nplay\n' | ./engine | python3 -c "
import sys, json
for line in sys.stdin:
    d = json.loads(line)
    print(d['msg'])
"
VibeStream engine ready.
Shuffle on.
Now playing: Starboy - The Weeknd (shuffled)
Now playing: Perfect - Ed Sheeran
Now playing: Believer - Imagine Dragons
```

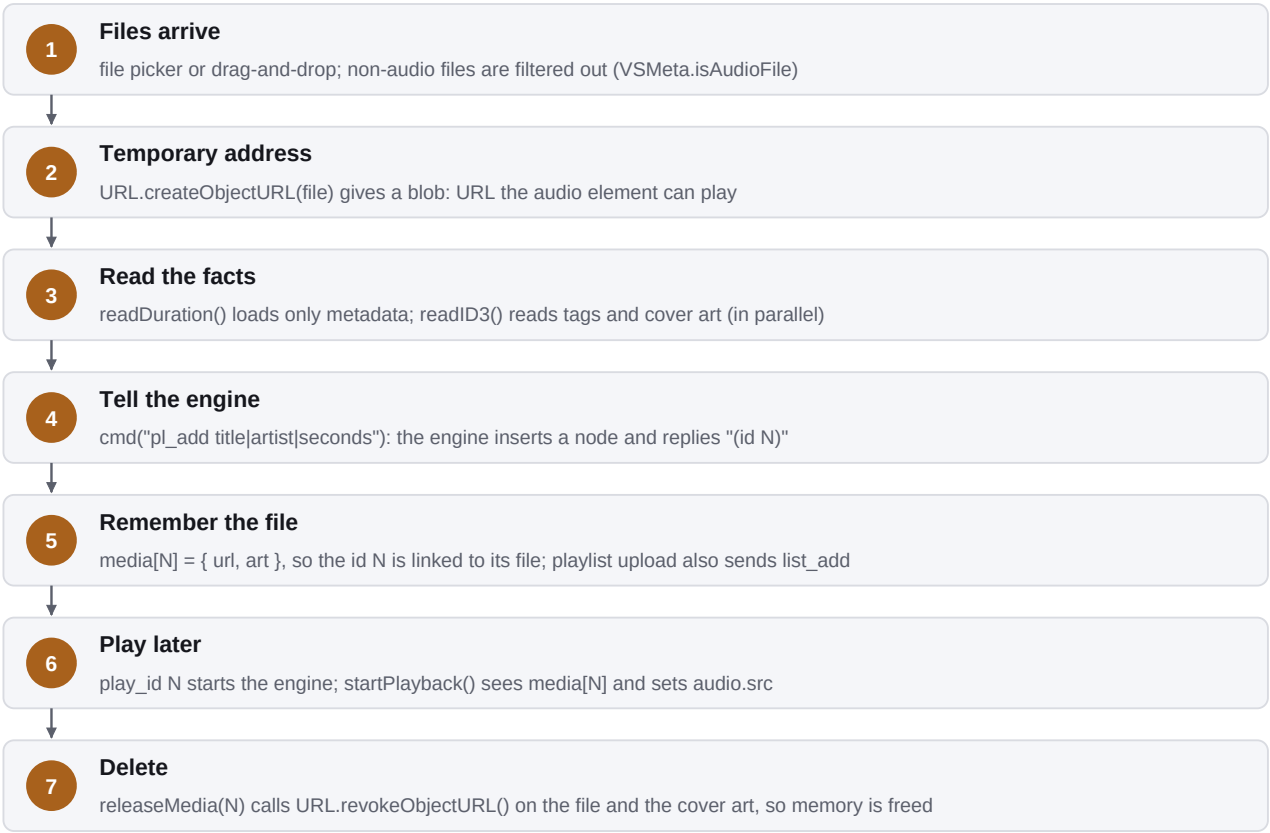
8. Key flows in more detail

8.1 Previous button and the history stack

Because every played song is pushed, **the top of the stack is the song playing now**. Previous therefore pops the top (discarding the current song) and *peeks* at the new top, which becomes the current song without being pushed again.

```
history (top first):   C  B  A       C is playing
press Previous      -> pop C, peek B
history:               B  A          B is playing  (B was NOT pushed again)
```

8.2 Uploading a file (browser side)



8.3 Reordering the queue

1. **Drop a song at a new place**
   dragover shows a line; drop computes (from, to) from the pointer position
2. **Send the move**
   cmd("q_move from|to")
3. **flatten()**
   copy priority queue + deque + queue into one array in display order
4. **Move the item**
   remove at from, insert at to; decide whether it stays "Play next"
5. **Safety check**
   array mode: refuse if either queue would exceed 10 songs
6. **Rebuild**
   clear the three structures, then pq_enqueue the "Play next" items and enqueue the rest

8.4 How shuffle and "play from a list" fit together

- `ctx` says which list is current. Without shuffle, `source_next` advances that list's cursor (and wraps when Repeat All made it circular).
- With shuffle, `shuffle_next` dequeues from the shuffle queue. The queue is rebuilt whenever the list changes (a `dirty` flag), excluding the song that is already playing so it does not repeat immediately.
- The Up Next structures always win: they are checked first on every Play.

9. Testing, memory safety and design decisions

9.1 How the project was checked

Check	What it proves
`gcc -Wall -Wextra` with zero warnings	No obvious C mistakes (unused values, bad types).
AddressSanitizer + UBSan run of the engine script	No out-of-bounds access, use-after-free or undefined behaviour while exercising lists, queues, shuffle and reorder.
macOS `leaks` on exit	0 leaks: every malloc has a matching free (including playlists, shuffle queue, history).
Python script driving the engine (dozens of assertions)	Shuffle plays each song once; playlists and liked songs behave; queue add / remove / move / clear; array cap; mode switch.
Node unit tests for meta.js	ID3 v2.3 / v2.4 parsing, UTF-8 / UTF-16 text, cover art, junk files do not crash.
Browser tests (Chrome DevTools Protocol)	Upload, drag and drop, real playback, seek, volume, delete + URL revoke, shuffle, playlist creation, queue drag / keyboard reorder, no JavaScript errors.

Note on the test scripts

The engine, ID3 and browser test scripts were run during development but are **not committed to the repository yet**. Adding them under a `tests/` folder is a good next step for the team.

9.2 Design decisions worth knowing

Decision	Reason
C engine owns all state; the UI only draws	Keeps the syllabus content in C and makes the UI replaceable (we redesigned it several times without touching the engine).
Return the whole state after every command	Simplest possible synchronisation: nothing can drift.
Two stack and two queue implementations, switchable at runtime	Lets you compare array and linked versions live in a presentation.
Array structures capped at 10	Matches the assignment ("MAX 10") and makes "full" visible.
Songs are copied into every list	Simple and safe (no shared pointers); cost: deleting a song must clean every list.
Shuffle builds a queue instead of picking random songs	Guarantees no repeats and reuses the queue concept.
Uploaded files are not persisted	Avoids storing user audio; a persistence layer (IndexedDB) would be a good extension.

9.3 Limitations and ideas for extension

- Uploaded tracks disappear on reload (could store files in IndexedDB).
- ID3 art / tags are read for MP3 only; other formats fall back to the file name.
- Priority queue enqueue / dequeue are O(n); a binary heap would make them O(log n) (good Module extension).
- Playlists cannot be reordered by dragging yet (only the Up Next queue can).
- The engine is single-user: all browser tabs share one state.

Where to look when something breaks

Wrong list contents -> engine.c (print the JSON with the command above). Wrong sound or time bar -> app.js section 6 (audio engine). Layout or colours -> style.css tokens at the top. Crash in C -> rebuild with `-fsanitize=address` and rerun the same commands.

10. Glossary and viva questions

10.1 Glossary

Term	Meaning
Node	One element of a linked structure: data plus pointer(s) to neighbours.
Head / tail	First / last node of a list.
NULL	The "points to nothing" pointer; marks the end of a non-circular list.
malloc / free	Ask the system for memory / give it back. Every node we create is freed on exit.
Cursor	A pointer remembering "where we are" in a list (the current song).
LIFO / FIFO	Last In First Out (stack) / First In First Out (queue).
Circular	The last element connects back to the first (a ring).
Priority queue	A queue where higher priority leaves first.
Deque	Double-ended queue: insert and delete at either end.
Modulo (%)	Remainder; used so array indexes wrap from the end back to 0.
Context (ctx)	The list we are currently playing from.
Flatten	Copy several structures into one array in display order.
JSON	Text format for structured data; the language of the engine's replies.
Blob URL	A temporary browser address pointing at a file in memory.
ID3 tag	Metadata (title, artist, cover) stored at the start of an MP3 file.

10.2 Likely viva questions, with short answers

Q1. Why is the playlist a *doubly* linked list?

We need Previous / Next and backward display; prev pointers also let us unlink a node without searching for its predecessor.

Q2. How does Repeat All work?

It makes the list circular: tail->next = head and head->prev = tail. Turning it off sets those two pointers back to NULL. Insert/delete temporarily break the circle to reuse the simple code.

Q3. Why use a stack for history?

The most recently played song must be the first one returned by Previous: that is exactly LIFO. Push on play, pop on Previous.

Q4. Why does Previous pop and then peek?

The top of the stack is the song that is playing now. Pop removes it, peek shows the one underneath, which becomes current without being pushed again.

Q5. What is the problem with a normal array queue, and how does a circular queue fix it?

Front moves right and the freed space at the start is never reused. Using index = (index + 1) % MAX wraps around so the space is reused.

Q6. How do you tell a circular queue is full or empty?

We keep a separate count: full when count == MAX, empty when count == 0 (this avoids the front == rear ambiguity).

Q7. How does "Play next" use a priority queue?

HIGH entries are inserted ahead of LOW ones and equal priorities stay first-come-first-served, so the first element is always the next song to play.

Q8. Array stack vs linked stack: which is better?

Array: simple and fast but fixed size (10). Linked: unlimited and uses memory only as needed but costs a pointer and a malloc per node. Both are O(1). The mode switch lets us demonstrate both.

Q9. How does shuffle guarantee no repeats?

It copies the list, shuffles the copy with Fisher-Yates, and puts it in a queue. Each Play dequeues one song, so every song appears once before the queue is rebuilt.

Q10. Why Fisher-Yates and not random swaps?

It gives every permutation equal probability in O(n); repeated random swaps are biased and slower.

Q11. How can the Up Next list be reordered when it is three structures?

We flatten the three into an array, move the element, then rebuild the structures. Removal works directly on the owning structure with removeAt.

Q12. What happens to a song in a playlist when it is deleted from the library?

forget_song() removes it from every playlist and from Liked Songs because each list stores its own copy.

Q13. What is the time complexity of inserting at the end of the playlist?

O(1), because the list keeps a tail pointer. Inserting at position p is O(p).

Q14. How do you avoid memory leaks?

Every malloc is paired with a free: delete functions free the node, *_free() walks and frees everything, and the engine/console frees all structures on exit. Verified with AddressSanitizer and leaks.

Q15. Why is the engine a separate C program instead of rewriting the logic in JavaScript?

The data-structure code is the graded part of the project; keeping it in C (nodes, pointers, malloc) means the web app demonstrates the same code as the console app.

Q16. How does the browser talk to C?

The Node server starts the engine as a child process and exchanges one text line per command on stdin / stdout; the engine answers with one JSON line.